

Design Report

Safe Semantic Planner in a Finite Cartesian State Space

PCCST503 - Machine Learning | Assignment 1

Department of Computer Science and Engineering

1. Introduction

This report describes the design and implementation of a safe semantic planner that operates in a finite Cartesian state space. The planner computes a safe path from an initial state to a goal state while avoiding designated bad (unsafe) states. The implementation uses A* search algorithm with Euclidean distance heuristic.

2. State Representation

Each state in the system is represented as a point in d-dimensional Cartesian space. A state has two attributes:

- id: A unique integer identifier for the state.
- embedding: A list of floating-point coordinates [x1, x2, ..., xd] representing the state's position in Cartesian space.

For our test cases, we use 2D embeddings (d=2), where each state has an (x, y) position. This allows the planner to calculate straight-line distances between states for the heuristic function.

State Class Definition

```
class State:
    id: int
    embedding: list # e.g., [0.0, 0.0] for 2D
```

3. Data Structures

3.1 Transition

A directed edge between two states. Each transition contains:

- id: Unique transition identifier.
- from_state, to_state: Source and destination state IDs.
- cost: Numeric cost of traversing this transition.
- safety: Safety score of the transition (0.0 to 1.0).
- reliability: Reliability score (0.0 to 1.0).
- available: Boolean flag indicating if the transition can be used.

3.2 Planning Problem

Bundles all inputs needed by the planner:

- initialState: ID of the starting state.
- goalState: ID of the target state.
- badStates: List of state IDs that must be avoided.
- states: List of all State objects in the environment.
- transitions: List of all Transition objects.

3.3 Planning Result

The output returned by the planner:

- success: Whether a valid path was found.
- statePath: Ordered list of state IDs from start to goal.
- transitionPath: Ordered list of transition IDs used.
- totalCost: Sum of all transition costs along the path.
- safetyScore: Minimum Euclidean distance to nearest bad state.
- exploredStates: Number of states explored during search.
- planningTime: Wall-clock time taken to find the path.
- reliabilityScore: Sum of reliability values of transitions used.
- score: Combined objective function value.

3.4 Internal Data Structures

During A* search, the planner uses the following internal structures:

- state_map (dict): Maps state ID to State object for O(1) lookup.
- adjacency (dict): Maps state ID to list of outgoing transitions (adjacency list).
- transition_map (dict): Maps transition ID to Transition object.
- open_set (min-heap): Priority queue ordered by f-score for A*.
- g_score (dict): Tracks cheapest known cost to reach each state.
- came_from (dict): Parent pointers for path reconstruction.
- closed_set (set): States already fully explored.

4. Heuristic Function

The heuristic function $h(n)$ estimates the cost from state n to the goal state. We use the Euclidean distance in the embedding space:

$$h(n) = \sqrt{(x1_n - x1_g)^2 + (x2_n - x2_g)^2 + \dots + (xd_n - xd_g)^2}$$

This heuristic is admissible (never overestimates) when transition costs are at least as large as the Euclidean distance between connected states. This guarantees that A* finds an optimal path with respect to transition cost.

5. Safety Computation

Safety is computed in two ways:

5.1 Bad State Avoidance

During search, the planner completely skips bad states. Any state in the badStates list is added to the closed set immediately and never expanded. This ensures the planner never visits a bad state.

5.2 Safety Distance

After finding a path, the planner computes the minimum Euclidean distance from every visited state on the path to the nearest bad state. This metric shows how close the path gets to dangerous areas:

```
safety_distance = min over all states s in path:
    min over all bad states b:
        euclidean_distance(s.embedding, b.embedding)
```

If there are no bad states, the safety distance is infinity (perfectly safe).

6. Objective / Scoring Function

The combined scoring function evaluates path quality:

```
Score(P) = alpha * G - beta * C + gamma * D + delta * R
```

Where:

- G = 1 if goal reached, 0 otherwise (default alpha = 10.0)
- C = total transition cost, lower is better (default beta = 1.0)
- D = min safety distance to bad states, higher is better (default gamma = 2.0)
- R = cumulative reliability of transitions (default delta = 1.0)

Higher scores indicate better paths. The weights can be tuned to prioritize cost minimization (increase beta), safety (increase gamma), or reliability (increase delta).

7. Algorithm: A* Search

The planner uses A* search, which works as follows:

1. Initialize the open set with the start state ($f = 0 + h(\text{start})$).
2. Pop the state with the lowest f-score from the open set.
3. If it is the goal, reconstruct and return the path.
4. If it is a bad state or already explored, skip it.
5. For each available outgoing transition:
 - a. Calculate tentative g-score = $g(\text{current}) + \text{transition.cost}$

- b. If this is better than the known g-score, update it
 - c. Compute $f = g + h(\text{neighbor})$ and push to open set
6. If the open set is empty and goal not reached, return failure.

8. Complexity Analysis

Time Complexity

$O((V + E) * \log V)$, where V is the number of states and E is the number of transitions. Each state is processed at most once, and each heap operation takes $O(\log V)$.

Space Complexity

$O(V + E)$ for storing the adjacency list, state map, g-scores, came_from pointers, and the open/closed sets.

9. Dynamic Environment and Replanning

The planner supports dynamic environments where the following may change:

- Goal state changes (Test Case 5)
- Transitions become unavailable (Test Case 4)
- New transitions are added (Test Case 6)
- Bad states change

Replanning approach: Modify the PlanningProblem object to reflect the change, then call `plan()` again. This is the simplest approach and works well for small to medium graphs. For very large graphs, incremental algorithms like D* Lite or LPA* could be used to reuse previous search results and avoid re-exploring unchanged portions of the graph.